CSE 2215: Data Structure and Algorithms-I

1 a) Distinguish between Array and Linked list.

ARRAY	LINKED LIST
Size of an array is fixed	Size of a list is not fixed
Memory is allocated from stack	Memory is allocated from heap
It is necessary to specify the number of elements during declaration (i.e., during compile time).	It is not necessary to specify the number of elements during declaration (i.e., memory is allocated during run time).
It occupies less memory than a linked list for the same number of elements.	It occupies more memory.
Inserting new elements at the front is potentially expensive because existing elements need to be shifted over to make room.	Inserting a new element at any position can be carried out easily.
Deleting an element from an array is not possible.	Deleting an element is possible.

b) What are the differences between static and dynamic memory allocation?

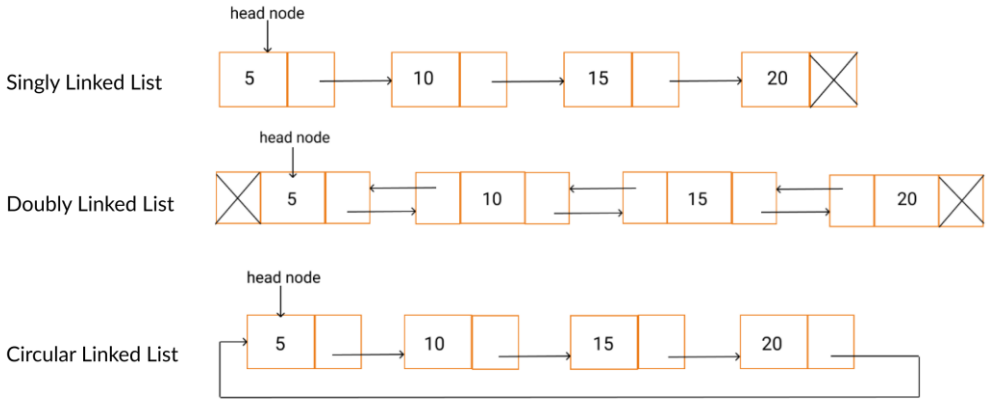
Answer: In static memory allocation, while executing a program, the memory cannot be changed. In dynamic memory allocation, while executing a program, the memory can be changed. Static memory allocation is preferred in an array. Dynamic memory allocation is preferred in the linked list.

2 a) Draw diagrams for a linear, doubly and circular linked list

b) Show memory mapping for each of the diagrams of Ques 2(a).

Answer:

a)



3 Show variable declarations for each of the structure type pointer variables given below

a) **student (name, id, marks, next)**
b) **head(data, next)**
c) **super(back, data, next)**
d) **start(row, col, data, right, down)**
e) **root(left, info, right)**

Where name is string type
id is int type
marks is float type
info is char type
row, col, data are int type
next represents address of next node
back represents address of previous node
right represents address of right node
down represents address of down node
left represents address of left node

Answers:

a)
```
struct student{
        char name[];
        int id;
        float marks;
        struct student *next;
};
typedef struct student student;
```

b)
```
struct head{
        int data;
        struct head *next;
};
typedef struct head head;
```

c)
```
struct super{
        int data;
        struct super *back, *next;
};
typedef struct super super;
```

d)
```
struct start{
        int row, col data;
        struct start *right, *down;
};
typedef struct start start;
```

e)
```
struct root{
        char info[];
        struct root *left, *right;
};
typedef struct root root;
```

4 **Suppose a linear linked list headed with "start" contains four nodes whose data values are 10, 20, 30, 40, respectively. Show the following operations**

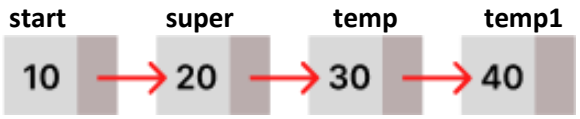
- **i) Draw a diagram for the linked list**
- **ii) What are the names of the nodes that contain 10, 20, 30, 40, respectively? Show the names of the nodes in the linked list diagram**
- **iii) Write statements to represent 10, 20, 30, 40, respectively**
- **iv) Write a statement to represent NULL at the end of the linked list**

Answers:

i)

ii)

```
struct list {
   int value;
   struct list *next;
};
typedef struct list node;
node *start, *super, *temp, *temp1;
```



iii)

Statements:

```
start = (node *) malloc (sizeof (node));
start->value = 10;
super = (node *) malloc (sizeof (node));
super->value = 20;
start->next = super;
temp = (node *) malloc (sizeof (node));
temp ->value = 30;
super->next = temp;
temp1 = (node *) malloc (sizeof (node));
temp ->next = temp1;
temp1->value = 40;
```

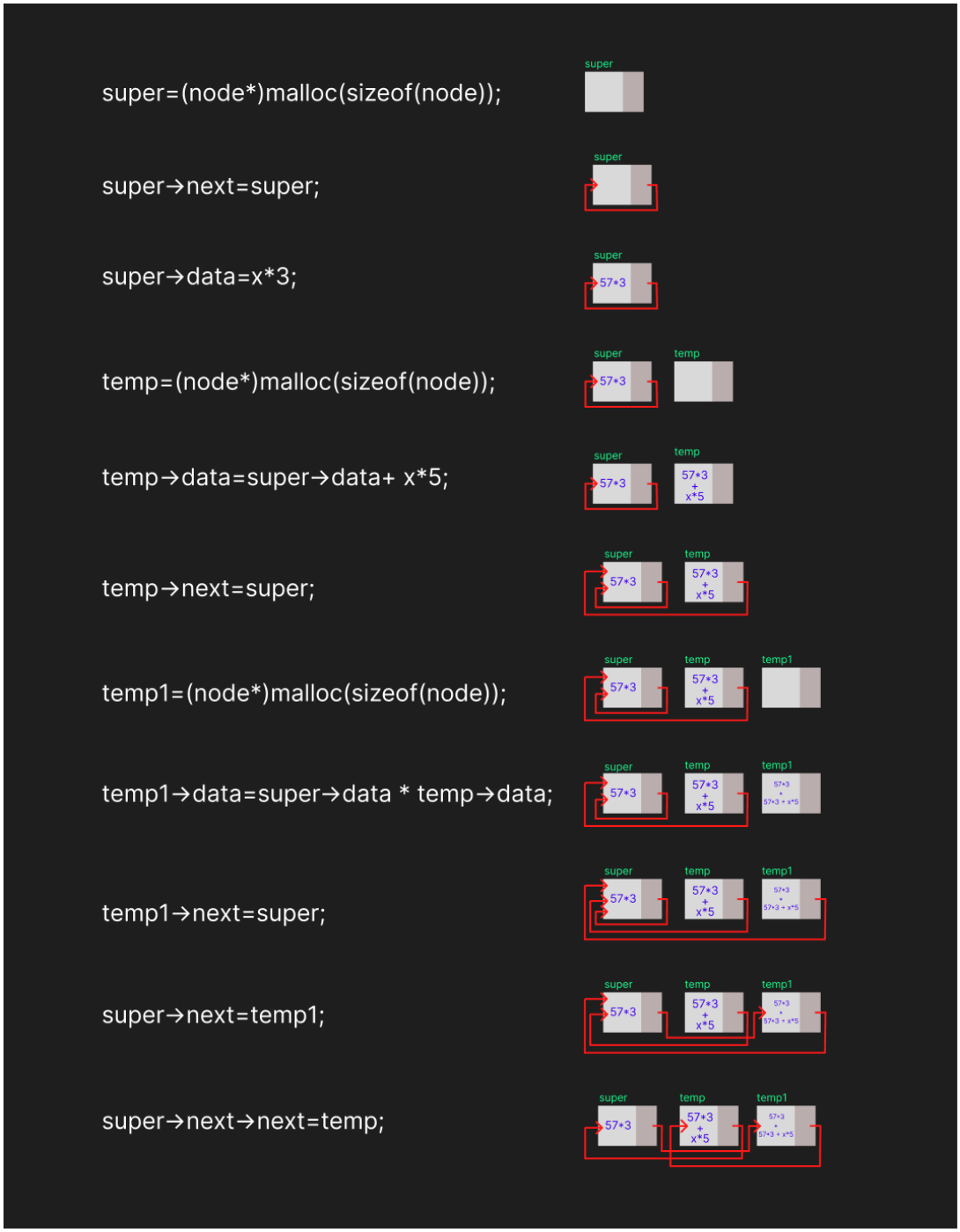
iv)

```
temp1->next = NULL;
```

5 **Show the effects of each of the following statements given using the following structure. Assume, x=57.**

```
struct list {
        int data;
        struct list *next;
      };
typedef struct list node;
node *super, *temp, *temp1;
```

Statements:



6 Write algorithms for the followings using the structure given below

```
struct list {
            int data;
            struct list *next;
          };
typedef struct list node;
```

a) To create a linear linked list in memory
b) To display the contents of a linear linked list
c) To insert an element in a linear linked list
d) To delete an element from a linear linked list
e) To concatenate two linear linked list
f) To reverse a linear linked list
g) To count the length of a linear linear linked list
h) To calculate the sum of all the data in the linked list

7 a) What are merits and demerits of doubly linked list over linear linked list?

b) Distinguish between linear and circular linked lists.

Answers:

a)

Key	Singly linked list	Doubly linked list
Complexity	In singly linked list the complexity of insertion and deletion at a known position is O(n)	In case od doubly linked list the complexity of insertion and deletion at a known position is O(1)
Internal implementation	In singly linked list implementation is such as where the node contains some data and a pointer to the next node in the list	While doubly linked list has some more complex implementation where the node contains some data and a pointer to the next as well as the previous node in the list
Order of elements	Singly linked list allows traversal elements only in one way.	Doubly linked list allows element two way traversal.
Usage	Singly linked list are generally used for implementation of stacks	On other hand doubly linked list can be used to implement stacks as well as heaps and binary trees.
Index performance	Singly linked list is preferred when we need to save memory and searching is not required as pointer of single index is stored.	If we need better performance while searching and memory is not a limitation in this case doubly linked list is more preferred.
Memory consumption	As singly linked list store pointer of only one node so consumes lesser memory.	On other hand Doubly linked list uses more memory per node(two pointers).

b)

The only difference between the singly linked list and a circular linked list is that the last node does not point to any node in a singly linked list, so its link part contains a NULL value.